LAB ASSIGNMENT 4-5

Advanced MLFlow and Data Versioning with DVC

Assignment Overview

Extend your reproducible ML project from Sessions 3–4. In Session 5, use MLflow to log model information, compare candidates, register model versions, and manage their lifecycle. In Session 6, use DVC to version datasets, configure storage, and recover a previous data version.

Core question: Can another engineer identify the model that was deployed and recover the code and dataset version used to create it?

```
Training runs → MLflow Tracking → Model Registry → Candidate/Staging → Production →
Archived/Retired
Dataset files → DVC metadata + Git → DVC remote storage → Recover selected data version
```

Learning Outcomes

- Explain model signatures, artifacts, autologging, and the MLflow Model Registry.
- Log and compare runs and register at least three model versions.
- Document a promotion workflow for candidate/staging, production, and archived/retired models.
- Use DVC to track dataset versions while Git tracks DVC metadata.
- Configure a DVC remote and push/pull data when storage is available.
- Restore an earlier dataset version and verify the result.
- Explain how Git, MLflow, and DVC complement one another.

Scenario and Dataset

You are an ML Engineer maintaining a house-price prediction or classification project. Several candidate models exist, and the team needs to know which model was evaluated, why one version was promoted, and which dataset version was used.

Reuse your Sessions 3–4 dataset if possible. Otherwise, use a small public or synthetic CSV dataset. Do not publish private, confidential, or sensitive data.

PART A LAB 4— ADVANCED MLFLOW

A1. Self-learning research

Use official MLflow documentation and other reliable sources. Explain each concept in your own words and include a practical example:

- Model Signature: describes expected model input/output schema and can help catch malformed inputs.
- Artifacts: files associated with a run, such as plots, reports, configuration files, or model files.
- Autologging: automatically logs supported parameters, metrics, and model information for supported ML libraries. Verify what your installed flavor actually records.
- Model Registry: manages registered model names, versions, metadata, descriptions, and lifecycle information.

- Version versus stage/alias: a version identifies a specific model revision; a stage or alias communicates intended use.

Version note: MLflow registry workflows vary by version and configuration. Some tutorials use Staging/Production stages; newer workflows may use aliases and tags. Check your installed version, state which workflow you used, and document equivalent candidate, production, and retired states if needed.

A2. Environment and project setup

```
python -m venv .venv
# Activate the environment for your operating system
python -m pip install --upgrade pip
python -m pip install mlflow scikit-learn pandas matplotlib
python -m pip freeze > requirements.txt
mlflow --version
```

Use your existing project structure where possible:

```
mlops-lab-4/
├── data/
├── src/ (train.py, evaluate.py)
├── models/
├── reports/figures/
├── configs/config.yaml
├── requirements.txt
├── README.md
└── .gitignore
```

A3. Track and compare experiments

1. Run your existing training script and verify that MLflow records the run.
2. Train at least three candidate models or variants with different hyperparameters.
3. Log relevant parameters and metrics. For classification, consider accuracy, precision, recall, and F1; for regression, consider MAE, RMSE, and R^2.
4. Log at least two artifacts, such as a confusion matrix, residual plot, feature-importance plot, configuration, or evaluation report.
5. Compare runs in the MLflow UI and select a candidate using an explicit evaluation criterion.

Explore autologging for your selected ML library. For example, Scikit-learn projects may use:

```
import mlflow
import mlflow.sklearn
mlflow.sklearn.autolog()
# Run your normal training and evaluation code here.
```

Inspect the run to verify what was logged automatically. Do not assume all information is captured.

A4. Add a model signature

6. Identify the expected model input columns and data types.
7. Prepare a representative, non-sensitive input example.
8. Log the model with a signature and input example using the method supported by your MLflow flavor/version.
9. Inspect the model details and document its expected input/output schema.
10. Explain how a signature can help detect malformed prediction requests and what it cannot guarantee.

Discussion: What could happen if a model expects four numeric columns but a future API sends three columns or changes a column type?

A5. Register model versions v1, v2, and v3

11. Select three trained runs and register their models under one registered-model name, such as HousePriceModel.
12. Verify that three distinct registered versions exist.
13. Record each version's source run, metrics, model changes, and intended use.
14. Add descriptions or tags where supported.
15. Take screenshots showing the registered model and its versions.

Version	Source run ID	Key metric(s)	Change/description	Decision
v1 / Version 1				
v2 / Version 2				
v3 / Version 3				

The labels v1/v2/v3 mean three registered versions, commonly displayed as Version 1, 2, and 3. They are not automatically Git tags or source-code versions.

A6. Lifecycle: staging, production, archived

Use the lifecycle mechanism supported by your MLflow installation. If stages are supported, demonstrate Staging, Production, and Archived. If your setup uses aliases/tags instead, document a clear equivalent such as candidate/staging tag, production alias, and archived/retired tag.

16. Define an acceptance rule before promotion, such as minimum F1 or maximum RMSE.
17. Mark one model as a candidate for validation.
18. Evaluate it against your rule and record the evidence.
19. Promote it or assign the supported production alias.
20. Mark a superseded model as archived/retired using the available stage or metadata mechanism.
21. Demonstrate how a teammate would identify and retrieve the intended production model.

Lifecycle state	Model version	Evidence/reason	Action
Staging / Candidate			
Production / Production alias			
Archived / Retired			

Critical thinking: Should the highest-accuracy model always be promoted? Consider class imbalance, error costs, fairness, latency, stability, and evaluation-data quality.

PART B —Lab 5: DATA VERSIONING WITH DVC

B1. Self-learning research

- Why Git is usually not ideal for large datasets or frequently changing binary files.
- What DVC tracks in Git and what is stored in the data remote.
- What a .dvc file represents.
- Difference between a Git commit and a DVC data version.
- What a DVC remote is and why teams need shared storage.
- How DVC supports reproducibility when data changes.

DVC complements Git: Git generally tracks code and small metadata files; DVC tracks data dependencies through metadata while large data files are stored in a local or remote cache.

B2. Install and initialize DVC

```
python -m pip install dvc
dvc --version
```

```
git status
dvc init
git status
```

Inspect the files DVC creates. Review and commit the generated metadata to Git. Explain why the metadata should be version-controlled.

B3. Track a dataset

Place a CSV file at data/dataset.csv, then run:

```
dvc add data/dataset.csv
git status
git add data/dataset.csv.dvc .gitignore
git commit -m "Track dataset with DVC"
```

The simpler root-level alternative is dvc add dataset.csv. Choose one layout consistently. Inspect the generated .dvc file and explain what it records. Verify that Git tracks the metadata rather than the large dataset contents.

B4. Configure storage and push data

Choose a local directory outside the repository for a basic exercise, or an approved shared/cloud remote for team collaboration. Never commit credentials or access tokens.

```
mkdir -p ../dvc-storage
dvc remote add -d localremote ../dvc-storage
git add .dvc/config
git commit -m "Configure DVC remote"
dvc push
```

A local remote is useful for learning commands on one machine but is not a shared cloud backup. For collaboration, follow DVC's current documentation for an approved shared remote and secure credential setup.

B5. Create and recover dataset versions

22. Record a baseline dataset version and commit its DVC metadata to Git.
23. Make a controlled change to the dataset, such as adding rows or correcting a value.
24. Run dvc add again, commit the updated .dvc metadata, and push the new data version.
25. Use Git history to identify the earlier commit containing the earlier .dvc metadata.
26. Restore that metadata in a safe working tree/temporary branch, then use the appropriate DVC checkout/pull command to restore the data.
27. Verify the restored file using row count, file size, checksum/hash, or a comparison with the baseline.
28. Return to the intended latest commit and verify that the latest data version can also be recovered.

```
# After modifying the dataset
dvc add data/dataset.csv
git add data/dataset.csv.dvc
git commit -m "Update dataset version"
dvc push

# Inspect history
git log --oneline
# Restore the matching DVC metadata from an earlier commit,
# then run the appropriate DVC checkout/pull command.
```

These commands illustrate the workflow, not a substitute for understanding Git state. Keep a clean working tree before switching versions and be aware that checkout operations can overwrite files. A temporary branch or Git worktree is a safer way to demonstrate recovery.

B6. Compare dataset versions

Version	Git commit	Rows	Columns	Change description	Restoration verified?
Dataset v1					
Dataset v2					
Dataset v3 (optional)					

Answer: How did you verify that DVC restored the intended version? Why is a filename alone insufficient to identify a dataset version?

B7. Explore DVC pipelines

As an extension, create a simple pipeline with data-preparation and model-training stages, or submit a one-page design. Identify each stage's inputs, outputs, code, parameters, and conditions for rerunning.

```
dvc stage add --help
dvc repro --help
dvc dag
```

Consult current DVC documentation for the syntax supported by your installed version. Explain how a pipeline output could be associated with an MLflow run.

PART C — INTEGRATE GIT, DVC, AND MLFLOW

29. Commit training code and DVC metadata to Git.
30. Make the required dataset version available through DVC.
31. Run training and log parameters, metrics, artifacts, and model with MLflow.
32. Record the Git commit ID and dataset version/hash in the MLflow run as tags or parameters where practical.
33. Register the selected model and document the promotion decision.
34. Explain how a teammate could recover the code, dataset, and model information needed to investigate a prediction.

Tool	What it tracks/records	Where information lives	Why it matters
Git	Source code and small metadata/configuration files		
DVC	Data versions and pipeline dependencies		
MLflow	Experiments, metrics, artifacts, model versions/metadata		

Expected repository structure

```
mlops-lab-4-5/
├── .dvc/
├── data/
│   └── dataset.csv.dvc
├── src/ (train.py, evaluate.py)
├── models/
├── reports/figures/
├── configs/config.yaml
├── requirements.txt
├── README.md
├── .gitignore
└── Git history containing data metadata versions
```

The actual dataset may be in the DVC cache/remote rather than committed as a normal Git file. Publish data only when its license and privacy status allow it.

Submission checklist and report format

- Report.pdf with objectives, setup, method, results, reflection, and references.
- GitHub repository URL and reproducible README instructions.
- requirements.txt or equivalent environment specification.
- MLflow screenshots showing runs, artifacts, registered versions, and lifecycle state/alias.
- Model comparison table and promotion rationale.
- DVC metadata files and Git history for at least two dataset versions.
- Evidence of push and previous-version recovery, if remote storage is available.
- Dataset version comparison table and Git/DVC/MLflow integration explanation.
- Challenges encountered and how they were resolved.

Title page and student details.
Objectives and brief theory.
Environment and tool versions.
tracking, signature, artifacts, and autologging.
Model Registry versions and promotion decision.
DVC initialization, tracking, remote, and recovery.
Integration workflow, screenshots, and tables.
Challenges, lessons learned, conclusion, and references.